

Universidad del Valle de Guatemala

Curso: Programación Orientada a Objetos

Segundo Ciclo, 2026

Laboratorio 2 - Arreglos y excepciones

1. Requisitos funcionales

El sistema deberá:

- Solicitar al iniciar el programa el nombre del caso, su código de identificación y el nombre del detective responsable.
- Crear un caso nuevo con todas las posiciones del arreglo de ubicaciones vacías y sin pistas registradas.
- Permitir reemplazar el caso actual por uno nuevo cuando el usuario seleccione la opción correspondiente.
- Administrar un máximo de cinco ubicaciones mediante un arreglo básico de objetos de tipo Ubicacion.
- Permitir registrar una ubicación en una posición específica del arreglo.
- Solicitar para cada ubicación su código, nombre, dirección o descripción del lugar, nivel de riesgo y estado.
- Verificar que la posición seleccionada para registrar una ubicación se encuentre dentro de los límites del arreglo.
- Impedir el registro de una ubicación en una posición que ya se encuentre ocupada.
- Validar que el nivel de riesgo de una ubicación se encuentre entre 1 y 10.
- Generar una IllegalArgumentException si se intenta crear una ubicación con un nivel de riesgo menor que 1 o mayor que 10.
- Consultar todas las ubicaciones registradas, mostrando la información de cada una y la posición que ocupa dentro del arreglo.
- Omitir durante la consulta general las posiciones del arreglo que contengan null.
- Consultar una ubicación específica mediante su posición dentro del arreglo.
- Informar al usuario cuando la posición consultada no exista o se encuentre vacía.
- Modificar el nivel de riesgo y el estado de una ubicación previamente registrada.
- Impedir la modificación de una posición del arreglo que contenga null.
- Validar nuevamente que el nivel de riesgo modificado se encuentre entre 1 y 10.
- Descartar una ubicación registrada, asignando nuevamente null a la posición que ocupaba.
- Impedir que se descarte una ubicación desde una posición inválida o vacía.
- Administrar una cantidad variable de pistas mediante un ArrayList<Pista>.

- Solicitar para cada pista su código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
- Validar que el nivel de importancia de una pista se encuentre entre 1 y 10.
- Validar que el nivel de confiabilidad de una pista se encuentre entre 0 y 100.
- Generar una `IllegalArgumentException` cuando la importancia o la confiabilidad de una pista no se encuentren dentro de los límites establecidos.
- Registrar nuevas pistas dentro del `ArrayList`.
- Impedir el registro de dos pistas con el mismo código.
- Consultar todas las pistas encontradas durante la investigación.
- Informar al usuario cuando todavía no existan pistas registradas.
- Buscar una pista mediante su código, recorriendo los objetos almacenados en el `ArrayList`.
- Informar al usuario cuando no exista una pista con el código solicitado.
- Modificar la información de una pista previamente registrada.
- Aplicar durante la modificación las mismas validaciones utilizadas al crear la pista.
- Eliminar del `ArrayList` una pista identificada por su código.
- Informar al usuario cuando se intente modificar o eliminar una pista inexistente.
- Calcular la cantidad de ubicaciones registradas.
- Calcular la cantidad de posiciones disponibles dentro del arreglo de ubicaciones.
- Determinar cuál ubicación posee el mayor nivel de riesgo.
- Calcular la cantidad de pistas registradas.
- Determinar cuál pista posee el mayor nivel de importancia.
- Determinar cuál pista posee el mayor nivel de confiabilidad.
- Calcular el promedio del nivel de importancia de todas las pistas registradas.
- Evitar la búsqueda de valores máximos o el cálculo de promedios cuando el `ArrayList` de pistas se encuentre vacío.
- Mostrar un reporte con todos los cálculos relacionados con las ubicaciones y las pistas.
- Presentar un menú con las trece operaciones solicitadas hasta que el usuario decida salir.

- Manejar mediante InputMismatchException los casos en los que el usuario ingrese texto u otro dato incorrecto cuando se espere un valor numérico.
- Limpiar la entrada incorrecta del Scanner después de una InputMismatchException.
- Manejar mediante IllegalArgumentException los valores numéricos que no cumplan las reglas de las ubicaciones o las pistas.
- Utilizar bloques try-catch para evitar que una entrada o un valor incorrecto finalice inesperadamente el programa.
- Utilizar al menos un bloque finally para realizar una acción final, independientemente de que la operación se complete correctamente o produzca una excepción.
- Mantener el programa en funcionamiento después de controlar una entrada incorrecta o una excepción.

Condiciones que permiten o impiden cada operación

Operación	Se permite cuando...	Se rechaza cuando...	Efecto
Crear un caso nuevo	Se proporcionan los datos necesarios del caso	Algún dato obligatorio es inválido o se produce un error de entrada	Se reemplaza el caso actual y se crean estructuras vacías
Registrar ubicación	La posición está entre 0 y 4, se encuentra vacía y los datos son válidos	La posición no existe, está ocupada o el riesgo no está entre 1 y 10	Se almacena una Ubicacion en la posición seleccionada
Consultar ubicaciones	Existe un caso actual	No existen ubicaciones registradas	Solo se muestran las posiciones que no contienen null
Consultar una ubicación	La posición existe y contiene una ubicación	La posición está fuera del arreglo o contiene null	Se muestra la ubicación sin modificarla
Modificar ubicación	La posición existe, contiene una ubicación y el nuevo riesgo es válido	La posición es inválida, contiene null o el riesgo no está entre 1 y 10	Se actualizan el riesgo y el estado
Descartar ubicación	La posición existe y contiene una ubicación	La posición es inválida o contiene null	La posición vuelve a contener null

Registrar pista	El código no está repetido y sus niveles son válidos	El código ya existe, la importancia no está entre 1 y 10 o la confiabilidad no está entre 0 y 100	La pista se agrega al ArrayList
Consultar pistas	El ArrayList contiene al menos una pista	El ArrayList está vacío	Se muestran todas las pistas sin modificar la colección
Buscar pista	Existe una pista con el código solicitado	Ningún objeto posee el código indicado	Se devuelve o muestra la pista encontrada
Modificar pista	La pista existe y los nuevos valores son válidos	El código no existe o algún valor está fuera de sus límites	Se actualiza la información de la pista
Eliminar pista	Existe una pista con el código indicado	La pista no se encuentra en el ArrayList	La pista se elimina de la colección
Mostrar reporte	Existe un caso actual	No se rechaza; se controlan por separado las estructuras vacías	Se muestran los resultados sin modificar los objetos
Salir	El usuario selecciona la opción 13	—	Termina la ejecución del programa

Casos especiales que se deben controlar

No se debe intentar utilizar los métodos de una posición del arreglo sin comprobar primero que no contenga null. Tampoco se debe acceder a una posición menor que cero o mayor que cuatro.

No se pueden registrar dos pistas con el mismo código. Para comprobarlo, se deberá recorrer el ArrayList antes de agregar una pista nueva.

No se debe buscar la ubicación con mayor riesgo si todas las posiciones del arreglo están vacías. De la misma forma, no se debe buscar la pista con mayor importancia, la pista con mayor confiabilidad ni calcular el promedio cuando el ArrayList esté vacío.

Una pista podrá tener una confiabilidad igual a 0 o 100, porque ambos valores forman parte del intervalo permitido. En cambio, los niveles de riesgo e importancia deben estar entre 1 y 10, incluyendo ambos extremos.

Si el usuario escribe texto cuando se solicita un número, el programa deberá capturar la `InputMismatchException`, limpiar el valor incorrecto almacenado en el `Scanner` y permitir que intente nuevamente.

Si el usuario proporciona un nivel fuera de los límites permitidos, el constructor o método modificador correspondiente deberá generar una `IllegalArgumentException`. Esta excepción será capturada durante la interacción con el usuario para mostrar el error y permitir que el programa continúe funcionando.

La creación de un caso nuevo reemplazará completamente al caso anterior. Por lo tanto, las ubicaciones y pistas del caso anterior dejarán de formar parte del sistema actual.

2. Clases propuestas

Clase Ubicacion

Representa uno de los lugares relacionados con la investigación del caso.

Cada objeto de esta clase almacenará la información de una ubicación específica. Además, será responsable de validar que su nivel de riesgo se encuentre dentro del intervalo permitido.

Sus responsabilidades son:

- Guardar el código que identifica la ubicación.
- Guardar el nombre de la ubicación.
- Guardar la dirección o descripción del lugar.
- Guardar el nivel de riesgo.
- Guardar el estado actual de la ubicación.
- Validar que el nivel de riesgo se encuentre entre 1 y 10.
- Permitir la modificación del nivel de riesgo y del estado.
- Generar una `IllegalArgumentException` cuando se intente asignar un nivel de riesgo inválido.
- Proporcionar la información de la ubicación para mostrarla al usuario.

La clase `Ubicacion` no necesita conocer la posición que ocupa dentro del arreglo. La posición será administrada por la clase `Caso`, ya que no es una característica propia del lugar, sino la ubicación que tiene dentro de la estructura de almacenamiento.

Clase Pista

Representa una evidencia encontrada durante la investigación.

Cada objeto de esta clase almacenará la información de una pista específica. También será responsable de validar que sus niveles de importancia y confiabilidad cumplan con los límites establecidos.

Sus responsabilidades son:

- Guardar el código que identifica la pista.
- Guardar la descripción de la pista.
- Guardar el tipo de evidencia.
- Guardar el nivel de importancia.
- Guardar el nivel de confiabilidad.
- Validar que el nivel de importancia se encuentre entre 1 y 10.
- Validar que el nivel de confiabilidad se encuentre entre 0 y 100.
- Permitir la modificación de la información de una pista.
- Generar una `IllegalArgumentException` cuando la importancia o la confiabilidad no se encuentren dentro de los límites permitidos.
- Proporcionar la información de la pista para mostrarla al usuario.

La clase Pista no será responsable de comprobar si su código ya existe. Esta validación corresponde a la clase Caso, porque es la clase que tiene acceso a todas las pistas almacenadas en el `ArrayList`.

Clase Caso

Representa la investigación que está siendo administrada por el sistema.

Esta será la clase administradora principal. Guardará los datos generales del caso y contendrá las estructuras utilizadas para almacenar las ubicaciones y las pistas.

Sus responsabilidades son:

- Guardar el nombre del caso.
- Guardar el código de identificación del caso.
- Guardar el nombre del detective responsable.
- Crear y administrar un arreglo básico con capacidad para cinco objetos de tipo `Ubicacion`.
- Crear y administrar un `ArrayList` de objetos de tipo `Pista`.

- Registrar una ubicación en una posición disponible del arreglo.
- Verificar que una posición se encuentre dentro de los límites del arreglo.
- Determinar si una posición contiene una ubicación o contiene null.
- Consultar todas las ubicaciones registradas.
- Consultar una ubicación específica mediante su posición.
- Modificar el nivel de riesgo y el estado de una ubicación existente.
- Descartar una ubicación y asignar null a la posición que ocupaba.
- Registrar pistas sin permitir códigos repetidos.
- Buscar una pista por medio de su código.
- Consultar todas las pistas almacenadas.
- Modificar una pista previamente registrada.
- Eliminar una pista del ArrayList.
- Calcular la cantidad de ubicaciones registradas.
- Calcular la cantidad de espacios disponibles en el arreglo.
- Encontrar la ubicación con mayor nivel de riesgo.
- Calcular la cantidad de pistas registradas.
- Encontrar la pista con mayor nivel de importancia.
- Encontrar la pista con mayor nivel de confiabilidad.
- Calcular el promedio de importancia de las pistas.

Cuando se cree un objeto de tipo Caso, el arreglo de ubicaciones tendrá exactamente cinco posiciones. Java inicializará automáticamente cada posición con null. El ArrayList de pistas se creará vacío.

La clase Caso concentrará las operaciones relacionadas con las estructuras de almacenamiento. De esta manera, Main no modificará directamente el arreglo ni el ArrayList, lo que permitirá conservar el encapsulamiento.

Clase Main

Será la clase encargada de la interacción con el usuario.

Sus responsabilidades son:

- Contener el método main.
- Crear la instancia inicial de Caso.

- Permitir que el usuario reemplace el caso actual por uno nuevo.
- Mostrar el menú con las trece opciones solicitadas.
- Solicitar al usuario los datos necesarios para cada operación.
- Leer datos de tipo texto, entero y decimal mediante Scanner.
- Invocar los métodos correspondientes de Caso.
- Crear objetos de tipo Ubicacion y Pista con la información ingresada.
- Mostrar los resultados de las consultas y operaciones.
- Capturar las excepciones generadas durante la creación o modificación de los objetos.
- Manejar la InputMismatchException cuando el usuario ingrese un dato no numérico.
- Limpiar la entrada incorrecta del Scanner.
- Utilizar un bloque finally en una operación apropiada.
- Mantener el menú en ejecución hasta que el usuario seleccione la opción de salir.
- Cerrar el objeto Scanner al terminar el programa.

La lógica relacionada con el almacenamiento, las búsquedas y los cálculos deberá permanecer principalmente en Caso. La clase Main se ocupará de recibir las entradas, controlar el menú y presentar los mensajes al usuario.

3. Propiedades de las clases

Atributos de Ubicacion

Atributo	Tipo	Visibilidad	Propósito
codigo	String	private	Identifica la ubicación dentro de la investigación. Permite distinguir un lugar de los demás al mostrar su información.
nombre	String	private	Almacena el nombre con el que se conoce la ubicación investigada.
direccion	String	private	Guarda la dirección o descripción que permite localizar e identificar el lugar.
nivelRiesgo	int	private	Indica el peligro asociado con la ubicación. Su valor debe encontrarse entre 1 y 10. También se utiliza para determinar la ubicación con mayor riesgo.

estado	String	private	Describe la situación actual de la ubicación, por ejemplo, pendiente, en investigación o investigada. Puede modificarse conforme avance el caso.
--------	--------	---------	--

El atributo nivelRiesgo requiere validación tanto al crear el objeto como al modificarlo. Si el valor es menor que 1 o mayor que 10, se generará una `IllegalArgumentException`.

La posición de la ubicación no se guardará como atributo. La posición corresponde al índice que el objeto ocupa dentro del arreglo administrado por Caso.

Atributos de Pista

Atributo	Tipo	Visibilidad	Propósito
codigo	String	private	Identifica de forma única una pista dentro del caso. Se utiliza para buscarla, modificarla y eliminarla.
descripcion	String	private	Explica en qué consiste la pista encontrada y proporciona la información necesaria para reconocerla.
tipoEvidencia	String	private	Clasifica la pista según el tipo de evidencia encontrada, como física, documental, digital o testimonial.
nivelImportancia	int	private	Representa la relevancia de la pista para la investigación. Debe encontrarse entre 1 y 10. Se utiliza para identificar la pista más importante y calcular el promedio de importancia.
nivelConfiabilidad	int	private	Indica qué tan confiable se considera la pista. Debe encontrarse entre 0 y 100. Se utiliza para determinar la pista con mayor confiabilidad.

Los atributos nivelImportancia y nivelConfiabilidad requieren validación al crear o modificar una pista. Los valores fuera de los intervalos permitidos generarán una `IllegalArgumentException`.

El código deberá ser único dentro del caso. Sin embargo, la comprobación de códigos repetidos será responsabilidad de Caso, porque esta clase tiene acceso a la colección completa de pistas.

Atributos de Caso

Atributo	Tipo	Visibilidad	Propósito
nombre	String	private	Almacena el nombre asignado a la investigación. Permite identificar el caso cuando se muestra su información o reporte.
codigo	String	private	Guarda el código de identificación general del caso.
detectiveResponsable	String	private	Almacena el nombre del detective encargado de la investigación.
ubicaciones	Ubicacion[]	private	Arreglo básico que almacena las ubicaciones del caso. Tendrá un tamaño fijo de cinco posiciones y podrá contener objetos Ubicacion o referencias null.
pistas	ArrayList<Pista>	private	Colección dinámica que almacena las pistas encontradas. Su tamaño podrá aumentar o disminuir cuando se registren o eliminen pistas.

Atributos de Main

La clase Main no necesita atributos de instancia obligatorios. Puede trabajar con variables locales dentro del método main, como en el programa anterior.

Variable local	Tipo	Propósito
scanner	Scanner	Permite leer los datos ingresados por el usuario.
casoActual	Caso	Guarda la referencia al caso que se administra en ese momento. Puede reemplazarse al seleccionar la opción "Nuevo caso".
opcion	int	Guarda la opción seleccionada dentro del menú principal.

Estas variables no son propiedades de los objetos de Main, porque únicamente se necesitan durante la ejecución del método main.

Estructuras de almacenamiento identificadas

Propiedad	Estructura	Objetos almacenados	Tamaño
ubicaciones	Arreglo básico Ubicacion[]	Objetos de tipo Ubicacion	Fijo: cinco posiciones
pistas	ArrayList<Pista>	Objetos de tipo Pista	Dinámico

El arreglo básico es adecuado para las ubicaciones porque el caso siempre tendrá un máximo de cinco espacios definidos. El ArrayList es adecuado para las pistas porque no se conoce con anticipación cuántas serán encontradas y algunas podrán eliminarse durante la investigación.

4. Métodos de las clases

Los métodos se organizarán de manera que cada clase sea responsable de las operaciones relacionadas con sus propios datos. Main manejará la interacción con el usuario, mientras que Caso administrará el arreglo y el ArrayList.

Métodos de Ubicacion

Método	Retorno	Parámetros	Propósito
Ubicacion(...)	Constructor	String codigo, String nombre, String direccion, int nivelRiesgo, String estado	Inicializa todos los datos de la ubicación. Antes de asignar el nivel de riesgo, comprueba que se encuentre entre 1 y 10.
getCodigo()	String	Ninguno	Devuelve el código que identifica la ubicación.
getNivelRiesgo()	int	Ninguno	Devuelve el nivel de riesgo. Será utilizado por Caso para encontrar la ubicación con mayor riesgo.
getEstado()	String	Ninguno	Devuelve el estado actual de la ubicación.
modificar(...)	void	int nuevoNivelRiesgo, String nuevoEstado	Valida y modifica el nivel de riesgo y el estado de la ubicación.
validarNivelRiesgo(...)	void	int nivelRiesgo	Comprueba que el nivel esté entre 1 y 10. Genera una excepción si el valor es inválido.
toString()	String	Ninguno	Devuelve todos los datos de la ubicación en un formato

			comprensible para mostrarlos al usuario.
--	--	--	--

El método validarNivelRiesgo(...) será private, porque solamente se utilizará dentro de Ubicacion. Los demás métodos serán public.

El constructor y el método modificar(...) podrán generar una IllegalArgumentException si reciben un nivel de riesgo menor que 1 o mayor que 10.

Métodos de Pista

Método	Retorno	Parámetros	Propósito
Pista(...)	Constructor	String codigo, String descripcion, String tipoEvidencia, int nivellImportancia, int nivelConfiabilidad	Inicializa los datos de una pista y valida sus niveles antes de almacenarlos.
getCodigo()	String	Ninguno	Devuelve el código de la pista. Se utiliza para buscarla, evitar duplicados, modificarla y eliminarla.
getNivellImportancia()	int	Ninguno	Devuelve la importancia de la pista. Se utiliza para encontrar la pista más importante y calcular el promedio.
getNivelConfiabilidad()	int	Ninguno	Devuelve la confiabilidad de la pista. Se utiliza para encontrar la pista más confiable.
modificar(...)	void	String nuevaDescripcion, String nuevoTipoEvidencia, int nuevaImportancia, int nuevaConfiabilidad	Modifica la información de una pista después de validar los nuevos niveles. El código se conserva como identificador.
validarNivellImportancia(...)	void	int nivellImportancia	Comprueba que la importancia se encuentre entre 1 y 10.

validarNivelConfiabilidad(...)	void	int nivelConfiabilidad	Comprueba que la confiabilidad se encuentre entre 0 y 100.
toString()	String	Ninguno	Devuelve toda la información de la pista en un formato adecuado para mostrarla.

Los métodos de validación serán private. Los constructores, métodos de consulta, modificación y toString() serán public.

El constructor y el método modificar(...) podrán generar una IllegalArgumentException si reciben una importancia o confiabilidad fuera de los límites permitidos.

Métodos de Caso

Constructor y datos generales

Método	Retorno	Parámetros	Propósito
Caso(...)	Constructor	String nombre, String codigo, String detectiveResponsable	Inicializa los datos generales del caso, crea el arreglo de cinco ubicaciones y crea el ArrayList vacío.
toString()	String	Ninguno	Devuelve el nombre, código y detective responsable del caso.

Administración de ubicaciones

Método	Retorno	Parámetros	Propósito
esPosicionValida(...)	boolean	int posicion	Comprueba que la posición proporcionada por el usuario se encuentre entre 1 y 5.
registrarUbicacion(...)	boolean	int posicion, Ubicacion nuevaUbicacion	Guarda una ubicación si la posición es válida y está disponible. Devuelve true si se registró y false en caso contrario.
obtenerUbicacion(...)	Ubicacion	int posicion	Devuelve la ubicación almacenada en la posición o null cuando la posición no es válida o está vacía.
consultarUbicaciones()	String	Ninguno	Recorre el arreglo y devuelve la información de todas las

			ubicaciones registradas, incluyendo sus posiciones.
modificarUbicacion(...)	boolean	int posicion, int nuevoRiesgo, String nuevoEstado	Busca la ubicación por su posición y modifica su riesgo y estado. Devuelve false si la posición es inválida o está vacía.
descartarUbicacion(...)	boolean	int posicion	Sustituye por null la ubicación almacenada en la posición indicada.
contarUbicaciones()	int	Ninguno	Recorre el arreglo y cuenta las posiciones que no contienen null.
contarEspaciosDisponibles()	int	Ninguno	Calcula cuántas de las cinco posiciones contienen null.
ubicacionMayorRiesgo()	Ubicacion	Ninguno	Recorre el arreglo y devuelve la ubicación con el mayor nivel de riesgo. Devuelve null si no hay ubicaciones registradas.

Administración de pistas

Método	Retorno	Parámetros	Propósito
registrarPista(...)	boolean	Pista nuevaPista	Agrega una pista al ArrayList si su código no se encuentra registrado.
buscarPista(...)	Pista	String codigo	Recorre el ArrayList y devuelve la pista cuyo código coincide. Devuelve null si no la encuentra.
consultarPistas()	String	Ninguno	Recorre el ArrayList y devuelve la información de todas las pistas. Informa si la colección está vacía.
modificarPista(...)	boolean	String codigo, String nuevaDescripcion, String nuevoTipoEvidencia, int nuevaImportancia, int nuevaConfiableidad	Busca una pista por su código y modifica su información. Devuelve false si no existe.

eliminarPista(...)	boolean	String codigo	Busca y elimina del ArrayList la pista que posea el código indicado.
contarPistas()	int	Ninguno	Devuelve la cantidad de objetos almacenados en el ArrayList.
pistaMayorImportancia()	Pista	Ninguno	Recorre la colección y devuelve la pista con mayor importancia. Devuelve null si no existen pistas.
pistaMayorConfiabilidad()	Pista	Ninguno	Recorre la colección y devuelve la pista con mayor confiabilidad. Devuelve null si no existen pistas.
promedioImportancia()	double	Ninguno	Suma la importancia de las pistas y divide el resultado entre la cantidad registrada. Devuelve 0 si el ArrayList está vacío.

Reporte

Método	Retorno	Parámetros	Propósito
generarReporte()	String	Ninguno	Reúne los cálculos solicitados y devuelve el reporte completo de la investigación. Controla los casos en los que no existan ubicaciones o pistas.

Este método utilizará los métodos de conteo, búsqueda de valores máximos y promedio. No modificará ningún objeto ni ninguna de las estructuras.

Métodos de Main

Método	Retorno	Parámetros	Propósito
main(...)	void	String[] args	Inicia el programa, crea el Scanner, solicita el caso inicial y controla el menú hasta seleccionar la opción 13.
crearCaso(...)	Caso	Scanner scanner	Solicita el nombre, código y detective responsable, y devuelve un nuevo objeto Caso.

mostrarMenu()	void	Ninguno	Muestra las trece opciones disponibles.
registrarUbicacion(...)	void	Scanner scanner, Caso caso	Solicita la posición y los datos de una ubicación, crea el objeto y trata de registrarlo.
consultarUnaUbicacion(...)	void	Scanner scanner, Caso caso	Solicita una posición y muestra la ubicación almacenada o el mensaje correspondiente.
modificarUbicacion(...)	void	Scanner scanner, Caso caso	Solicita la posición, el nuevo riesgo y el nuevo estado.
descartarUbicacion(...)	void	Scanner scanner, Caso caso	Solicita la posición y trata de eliminar la ubicación del arreglo.
registrarPista(...)	void	Scanner scanner, Caso caso	Solicita los datos, crea una pista y trata de agregarla al ArrayList.
buscarPista(...)	void	Scanner scanner, Caso caso	Solicita un código y muestra la pista correspondiente.
modificarPista(...)	void	Scanner scanner, Caso caso	Solicita el código y los nuevos datos de una pista.
eliminarPista(...)	void	Scanner scanner, Caso caso	Solicita un código y trata de eliminar la pista correspondiente.
leerEntero(...)	int	Scanner scanner, String mensaje	Lee un número entero y controla entradas incorrectas mediante InputMismatchException.
leerTexto(...)	String	Scanner scanner, String mensaje	Solicita y devuelve una línea de texto sin espacios innecesarios al principio o al final.

Los métodos auxiliares de Main serán private static, porque solamente se utilizarán dentro del driver program.

Operaciones que pueden producir excepciones

Operación	Excepción	Causa
Leer una opción, posición o nivel	InputMismatchException	El usuario escribe un valor que no es un número entero
Crear una ubicación	IllegalArgumentException	El riesgo no se encuentra entre 1 y 10
Modificar una ubicación	IllegalArgumentException	El nuevo riesgo no se encuentra entre 1 y 10
Crear una pista	IllegalArgumentException	La importancia o la confiabilidad están fuera de sus intervalos
Modificar una pista	IllegalArgumentException	Uno de los nuevos niveles es inválido
Acceder al arreglo sin validar	ArrayIndexOutOfBoundsException	Se intenta utilizar una posición inexistente
Utilizar una posición vacía	NullPointerException	Se intenta llamar un método sobre una posición que contiene null

Las últimas dos excepciones deberán prevenirse mediante condiciones antes de acceder al arreglo o utilizar el objeto. Las primeras dos clases de excepciones sí formarán parte explícita del manejo mediante try-catch.